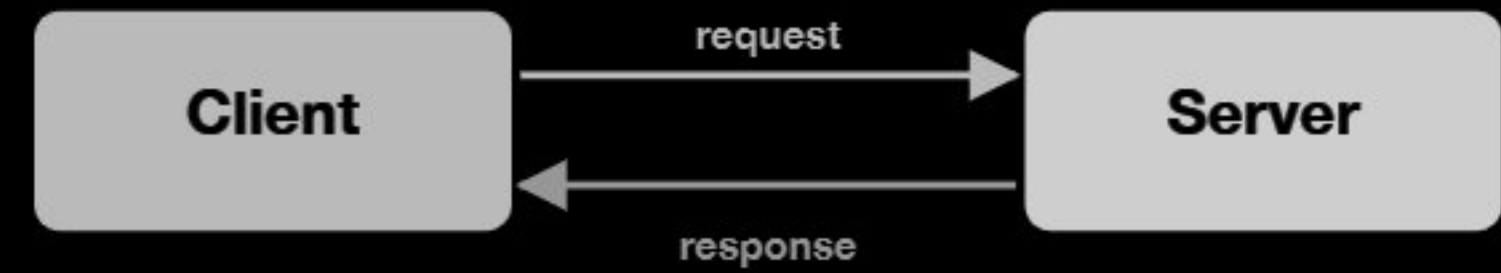


The Journey of a Click

Full Stack: Frontend, Backend, Database & Deployment · Class 1

Client & Server

- A client asks; a server answers
- Your laptop is a client when it asks Google for a page
- Your laptop is a server if it answers another machine's request
- "Client" and "server" are roles, not fixed identities



Names vs. Addresses: Why DNS?

- Every machine on the internet is found by its numeric IP address
- Humans remember words far better than numbers
- But IPs change too — servers move, load balancers rotate, providers reassign them
- So even machines need a stable name; DNS's job is keeping that name fixed while the IP behind it changes

`hotstar.com` → `99.86.30.53`

DNS Server & Your ISP

- DNS = Domain Name System — the internet's phonebook
- Your ISP (Internet Service Provider) is what connects you to the internet at all
- Your device usually asks your ISP's DNS resolver first
- That resolver list isn't fixed on your machine — it's handed to you via DHCP each time you join a network (`scutil --dns` / `ipconfig` just shows what your router/ISP gave you)

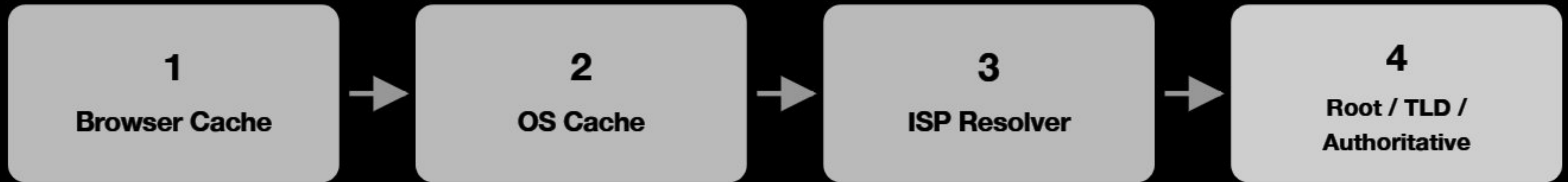
IPv4 vs IPv6

- IPv4: 32-bit, written as 4 dotted numbers — e.g. `192.168.1.1`
- IPv4 has ~4.3 billion possible addresses (~430 crore) — and we ran out
- IPv6: 128-bit, written in hex groups — e.g. `2401:4900:50:9::2e0`
- IPv6's address space is vast enough for every device, forever

Private & Special IP Addresses

- `192.168.x.x` is a private address — only meaningful on your local network
- Every home router hands out private addresses like this to its devices
- `8.8.8.8` is Google's public DNS resolver — a well-known, memorable IP
- Pinging a raw IP skips the DNS lookup step entirely

The DNS Resolution Chain



checked in order — stop as soon as one has the answer

HTTP: Request & Response

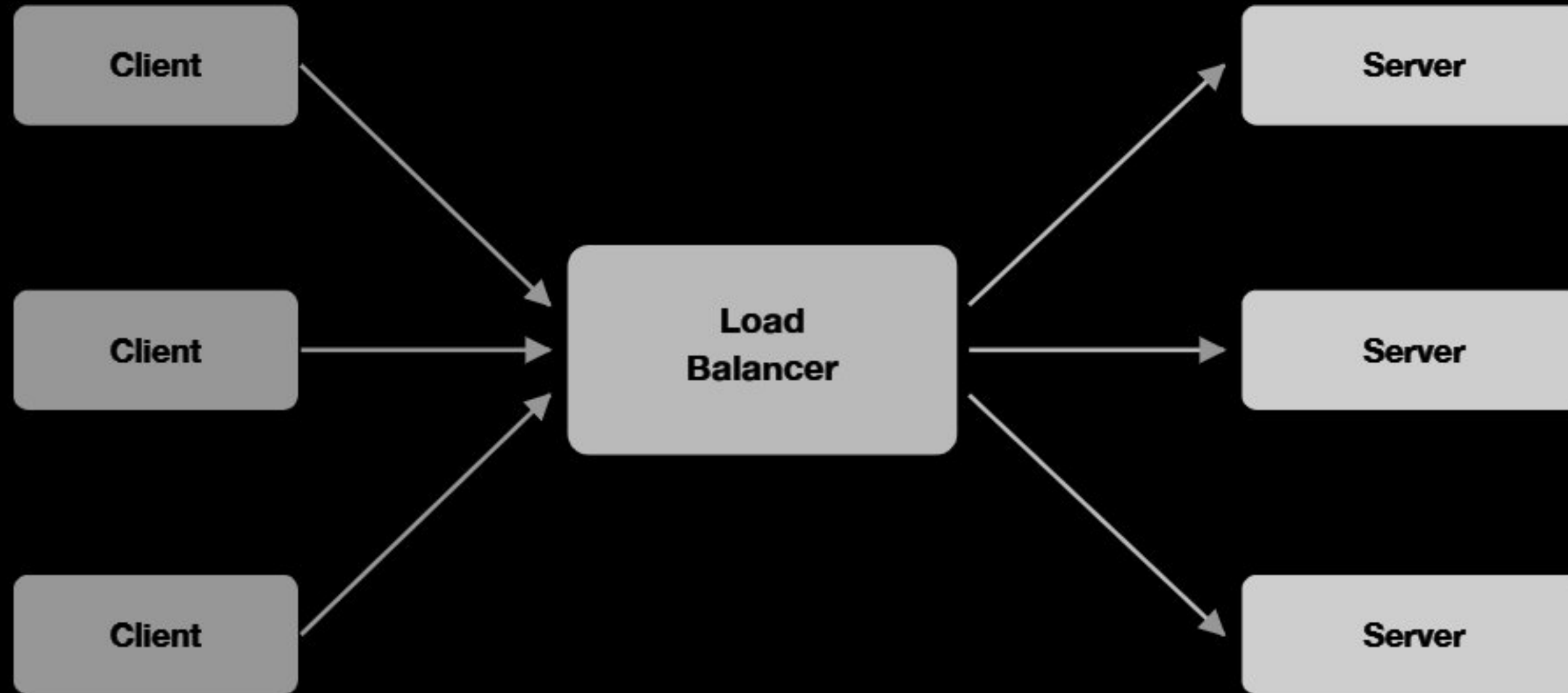
- Once the browser has the IP, it opens a connection to that server
- It sends a request: "GET me the homepage"
- The server sends back a response — plain text, not a picture
- The response from the Frontend Server is usually a HTML



HTML, CSS & JavaScript

- **HTML** — the structure (the skeleton of the page)
- **CSS** — the style (the skin and clothing)
- **JavaScript** — the behavior (the muscles and nerves)
- All three arrive as plain text files from the server

Handling Millions of Users: Load Balancing



The Browser's Job

- Downloads the HTML, CSS, and JS text files
- Parses HTML into a DOM (Document Object Model) — a tree of the page — then paints pixels on your screen
- A browser is a **rendering** engine — it turns text into a page
- Chrome (Google), Safari (Apple), Firefox (Mozilla), Edge (Microsoft)

Static vs. Dynamic Content

- A static page looks the same for every visitor
- A dynamic page (your cart, your orders) is different per user, per request
- Dynamic answers need a program running on the server — the backend
- The backend computes the answer fresh for each request

APIs: Frontend Talks to Backend

- The frontend (HTML/CSS/JS in your browser) can't reach the backend on its own
- JavaScript is the part of the frontend that makes the request
- An API is the agreed-upon doorway the frontend calls through
- Think of it as the connective tissue between frontend and backend



JSON: The Data Format

- The backend should send data that is readable by a human (developer) as well as by a machine, programmatically
- JSON: data written as simple key/value pairs
- Example: `{"name": "Asha", "age": 21}`
- Almost every API sends and receives JSON

Why Data Loads in Pages

- Amazon has millions of products — sending them all at once would be huge
- Large responses are slow to send and slow for your browser to handle
- So the backend sends one page's worth at a time
- You trigger the next page only when you actually need it

RAM vs. Disk & Why Not the App Server

- RAM is fast but volatile — its contents vanish when power is lost
- Disk (HDD/SSD) is slower but persistent — data survives a restart
- User data (logins, orders, cart) must survive — so it belongs on disk
- Storing it inside the app server itself doesn't scale past one machine

What a Database Does

- Stores data safely and persistently, long after a request ends
- Retrieves exactly the data a request asks for, quickly
- Organizes data with structure and keeps records consistent
- Secures data and manages relationships between records

The Full Journey

